

Evaluating Active-learning Based Performance Prediction of Parallel Applications

Shivam Aggarwal[§]

Oracle India Pvt. Ltd., India

Preeti Malakar

Indian Institute of Technology Kanpur, India

Abstract—Many scientific applications use Message Passing Interface for distributed-memory parallelism. These applications simulate complex physical phenomena and are routinely executed on supercomputers and high performance compute clusters. The applications may run for several hours or days and there may be long queue waiting times on the supercomputers. Often the application developers are unable to correctly estimate the runtime of their application for a new configuration and thus may overestimate the runtime requirements. This may increase the queue wait times along with sub-optimal job scheduling decisions. Thus accurate prediction of the performance of an application (performance modeling) is helpful. We have developed statistical models for performance prediction of MPI applications based on active learning algorithms and are able to achieve low Median Absolute Percentage Error (MdAPE) in many cases. We used five scientific applications and benchmarks – HPCG, LULESH, miniAMR, miniMD, and miniFE. We predict the execution times of these applications using various data sizes on up to 512 cores of the PARAM Sanganak supercomputer at IIT Kanpur. The statistical models gave MdAPE of 10.6, 3.5, 8.21, 12.62, and 32.25% for HPCG, LULESH, miniAMR, miniMD, and miniFE respectively.

I. INTRODUCTION

Message Passing Interface (MPI) [1] is the standard for developing distributed-memory parallel applications. Various scientific domains use MPI to develop applications to perform large-scale simulations and complex calculations. These applications require extensive computing resources, available on large supercomputers or HPC clusters and often shared by multiple users. Users submit their jobs requesting for a desired number of nodes and wall-clock time. Performance prediction [2], [3], [4], [5] is often used to estimate an application's runtime based on application and system features. Accurate performance prediction is challenging because the application's runtime depends on various factors such as the number of nodes, number of MPI processes per node, problem size, network topology, and network congestion.

Several thousands of users submit their jobs on high performance computing (HPC) clusters and supercomputers. A typical job is submitted with specifications regarding the required node count, process per node, memory requirement, the time required, and application-specific parameters. In most cases, the actual time needed by the job differs from the requested time. This leads to poor system utilization because some jobs may be considered for longer duration than required and thus may disrupt the job schedules [6], [7], [8]. One of the

primary reasons is the absence of accurate performance prediction. Performance modeling also help developers understand scalability of MPI applications.

Prior work in performance prediction uses two approaches: analytical [9], [10], [11], [12] and statistical modeling [13], [14], [15], [16]. Analytical models are primarily mathematical models that require in-depth knowledge about application behavior, communication patterns, memory requirements, etc. The main advantage of analytical modeling is that it does not require any training time or data. Analytical models are developed considering the application's communication patterns and memory requirements. Statistical models are based on classical machine learning approaches [17] that require a large training dataset to effectively learn the mapping function between input and output domains. However, obtaining a large amount of training data is costly for HPC performance prediction because we need to execute the applications and record the execution times of the applications (labels). This also incurs significant queue waiting times for the compute resources thus increasing the training cost. We may be able to use historical job logs for an application if the same input configurations were used for executions and we have sufficient data available for varying process counts.

In this work, we have developed statistical model based on active learning (AL). Our approach is generic; the model is not specific to any particular application. This is because the models are based on the output of parallel profilers. Our statistical models are based on the active learning [18] methodology, where data samples are strategically chosen [16]. Instead of obtaining the labels for all data points (here, the application's execution times), we obtain labels (execution time) only for the selected samples to reduce the training time. We have implemented the greedy sampling algorithms – G_x , G_y , and G_{xy} proposed by Wu et al. [19], [20] and ALER algorithm [21] based on the Query by committee (QBC) approach.

Greedy sampling and QBC are the two major strategies used in active learning. Greedy sampling aims to introduce diversity in the final dataset to improve the machine learning algorithm's performance. The G_x algorithm introduces variety in the input domain, while the G_y algorithm focuses on the output domain. The G_{xy} algorithm combines the input and output domain and introduces diversity in the combined domain. The QBC approach builds a committee of different machine learning algorithms and selects the new samples based on the disagreement of the committee members. We have used three

[§]Work done when the author was a student at IIT Kanpur

ensemble-based machine learning algorithms – Random Forest [22], Extra Trees Regression [23], and XG Boost [24] in our work based on prior work [25], [26], [15].

Our active learning based predictions achieve low values of Median Absolute Percentage Error (MdAPE) on the test data. We tested our performance models on five scientific applications and benchmarks, namely HPCG [27], LULESH [28], and Mantevo benchmarks– miniAMR, miniFE, and miniMD. We trained our statistical models on 40 and 60% of training data. Our statistical models predict the execution time of HPCG, LULESH, miniAMR, miniMD, and miniFE with the best MdAPE of 10.60, 3.55, 8.21, 12.62 and 32.25.

II. RELATED WORK

Performance prediction is an active research area in high performance computing (HPC). Many researchers have proposed various analytical and statistical methods for performance modeling and prediction. Culler et al. [9] proposed the LogP model in the early 1990s, which is considered the first analytical model for performance modeling. This was based on four parameters – communication latency (L), overhead (o), bandwidth (g), and the number of processors (P). Alexandrov et al. [10] proposed LogGP, an improved version of the LogP model. It has an additional parameter G that specifies the bandwidth obtained for larger messages. The LogGP model improved the performance of the LogP model for larger message sizes. These models are mainly used for calculating the point-to-point performance of MPI communications on the various systems. We have focused on predicting the total computation time and communication time of MPI applications, which includes both point-to-point and collective MPI communications. Several works [29], [13], [30] focus on analytical models that requires in-depth understanding of the application as well as knowledge about the application communications. In contrast, we focused on statistical learning based performance prediction where detailed application knowledge is not required.

Many researchers have used machine learning (ML) techniques for parallel performance prediction [25], [15], [31]. These techniques heavily rely on understanding the relationship between the input and output domain; specific application knowledge is not required to develop statistical models. Sun et al. [14] proposed a framework for automated prediction of the execution time of MPI programs. They use an ML model with features such as variable assignments, branch information, loops, and MPI communications. Hunold et al. [15] used performance prediction to solve the algorithm selection problem in MPI implementations. They trained several regression models to predict the runtime of various algorithms and then picked the algorithm that yields the minimum runtime among all algorithms. Singh et al. [16] experimented with different machine learning-based approaches to predict the performance of parallel applications. Their focus is on training a model with minimal data, and they have experimented with techniques such as active learning and ensemble. Majority of the previous research focuses on classical machine learning in performance

prediction, where they need to generate a large dataset for training the models.

Many techniques such as transfer learning [32], active learning [18], and few-shot learning [33] are used in the literature to handle the problem of limited training data. We aim to develop performance prediction models for massively parallel programs using active learning methods with less training data. Our work is inspired from the active learning techniques proposed by Dongrui et al. [19], [20] and Fazakis et al. [21]. Agarwal et al. [26] perform auto-tuning of parallel I/O parameters such as Lustre stripe size and stripe count, and MPI IO tuning parameters such as collective I/O aggregator buffer size. They use Bayesian optimization to minimize their objective function and find the best set of parameters. Our work is orthogonal from their work, we have evaluated sampling techniques that help to learn from less data.

III. STATISTICAL PERFORMANCE MODELS FOR PARALLEL (MPI) PROGRAMS

A parallel application performs computation, communication, and I/O. The application first distributes the data to different MPI ranks (domain decomposition), and then all ranks perform computations in parallel followed by intermittent communications between processes and I/O. The total execution time of a parallel application is the sum of its communication time, computation time, and I/O time. However, it is not straightforward to model this analytically because there may be partial overlap among them due to asynchronous executions. The major bottleneck in using machine learning algorithms for performance prediction is obtaining a comprehensive training dataset. We need to schedule the application to run on the desired system to find the application's runtime for different configurations. We cannot use these systems to schedule runs for long duration due to limited compute times available on some systems and high queue waiting times. We used active learning with the goal of learning from fewer samples. Active learning is a subfield of machine learning where the learning algorithm interactively selects the training samples [20], [21]. We employ active learning because we want the algorithms to select the training data. The algorithms select the data on which it is not confident (i.e. error is maximum). Active learning is helpful in those scenarios where we have enough unlabeled training data but labeling the training data is very expensive. We can quickly generate the unlabeled training data, and as the algorithm demands, we can select the chosen sample to run on the desired system to obtain the runtime (output).

We formulate the performance prediction problem as a regression problem in machine learning, where the goal is to predict the application's execution time based on a set of parameters. The execution time of an application on an HPC cluster/supercomputer depends on many factors such as the congestion in the network, unexpected delays due to other tasks, the number of nodes and processors used, application-specific parameters such as data size. We have used the following parameters to model – number of nodes, number of processes per node, and data size (grid size in x, y, and z dimensions).

These are the input variables for regression, and execution time is the target variable. We used pool-based sequential active learning (AL) for our models. We sequentially select one label at a time from the pool of unlabeled training data. A pool-based sequential algorithm works by repetitively adding a data sample to the training dataset until the desired size of the training set is achieved. The active learning algorithm selects a sample from the training pool using some criteria such as the sample on which the trained model shows least confidence or maximum disagreement of committee members. Then we obtain the actual label (actual runtime) for this sample. We add this sample to the training set and re-train our machine learning model. The active learning algorithm uses this model to select the subsequent sample for the training dataset. We have developed four statistical models to predict the runtime of MPI applications. These are based on two active learning strategies – Greedy sampling (GS) [20] and Query by committee (QBC) [21]. We now explain the models.

A. Greedy Sampling

An active learning algorithm should select a new sample so that it adds value to the existing training dataset. Selection of new sample is crucial because it heavily influences the active learning algorithm's performance. Informativeness, representativeness, and diversity are some parameters that effect sample selection. Greedy sampling techniques [20] mainly focus on diversity while including a new sample to the existing training dataset. It also considers the geometric properties of the input and output domains while selecting a new sample. We have experimented with three different greedy sampling techniques for our performance prediction problem. These algorithms work in two stages. Some initial samples are added to the training dataset in the first stage, and their labels are obtained. We have used a different method of initial selection of samples than the original algorithm. The algorithms proposed earlier [20] select the initial samples distributed among the pool and farthest from the centroid of the unlabeled pool. We have chosen the initial samples (say k) closest to the centroid of the unlabeled pool. We found this approach to perform better. This is because later samples are included in order to increase the diversity. Thus the initial training data contain samples both close and distant from the centroid. After building an initial dataset, subsequent samples are selected in the next stage. We explain our three greedy sampling algorithms in the following subsections.

1) *GS on the input domain (G_x)*: This algorithm focuses on increasing the diversity in the input space. It is independent of the regression model used as it does not update the regression model after adding a new sample to the training set. G_x algorithm is considered a passive algorithm due to this property. Initially, k samples are chosen that are closest to the centroid of the pool and are added to the training dataset. For each sample in the pool, we calculate the distance of the sample from the centroid of the selected dataset, and the sample with the maximum distance is added to the training dataset. This is done so that the sample set is diverse. We use the Manhattan distance to find the distance between a sample and the centroid.

When a sample is added to the training set, it is removed from the pool. When the desired size of the training set is achieved, we obtain the labels for the selected samples one by one. After this, we train regression model on this dataset.

2) *GS on the output domain (G_y)*: G_y algorithm focuses on introducing diversity in the output domain of the dataset. For this, we need to know the output of all samples to evaluate the diversity. The actual output is unknown for the samples, therefore we used the predicted output by the regression model to evaluate the diversity. We require a regression model in the beginning to execute the G_y algorithm. Similar to G_x , we choose k samples that are closest to the centroid of the pool, and obtain the labels (execution time) for these samples. We train our regression model on this dataset. The number of initial samples ' k ' is an important factor in the performance of the algorithm. We need some samples to train our initial regression model, and the accuracy of the regression model depends on the number of samples. If we keep a large number of samples in the initial set, then we will not be able to diversify our final training set, as all these samples are closest to the centroid of the training pool. We have used only 5 samples in our initial set. Smaller number of labels can be obtained quickly to start the algorithm and we keep improving the regression model when new samples are added to the training set in each iteration. The G_y algorithm aims to select the sample whose output is farthest from the centroid of already selected samples.

3) *GS on input and output (G_{xy})*: G_x focuses on increasing diversity in the input space, while G_y focuses on increasing diversity in the output space. In G_y , the regression model is trained from a minimal number of samples for the initial stage, so it may not be accurate. To overcome this problem, we also experimented with a combination of G_x and G_y [20]. The focus of G_{xy} algorithm is to increase diversity in both input and output domains. G_{xy} algorithm defines a new distance metric d_{xy} to find the distance between a sample and the centroid of the current training set. Let d_x and d_y be the distances used in the G_x and G_y algorithms, then $d_{xy} = d_x \times d_y$. The reason to choose multiplication of d_x and d_y is that they both may have different scales and multiplication is not affected by the different scales of quantities. G_{xy} algorithm works similar to G_y except for the change in distance parameter. Here too, we select the next sample that has the maximum distance in order to increase diversity.

B. Query by Committee (QBC)

Query by Committee (QBC) [34] is a popular strategy in the active learning literature. We have used this for performance prediction of parallel programs. We predict with a regression model on the unlabeled pool to select a new sample in the training dataset. As we do not have enough data to train a perfect model in the early stages, we do not wholly rely on the model's prediction. In the QBC approach, we use a committee of models instead of a single model. We experimented with an ensemble-based active learning regression algorithm (ALER) [21] that is based on evaluation of uncertainty of committee

members on a sample. We explain our methodology of using QBC for performance prediction problem next.

The algorithm assumes that we have an initial training set with true labels (execution time). We have used a similar method as Greedy sampling to build the initial training set. We start the algorithm with an empty dataset, and calculate the centroid of the unlabeled pool. We select k initial samples that are closest to the centroid and obtain the execution time for these samples. The committee contains three regression models, so we initialize three models c_1 , c_2 , c_3 , and train these on the initial dataset. This process is repeated until we have the desired number of samples in the dataset. We use our regression models c_1 , c_2 , and c_3 to predict the execution times for each sample in the unlabeled pool. ALER defines a metric named RPO (range of predicted outputs) to measure the uncertainty of predictors. For each sample, RPO is calculated as the difference between the maximum and minimum predicted value by committee members. We select the sample with the maximum RPO to be included in the training dataset and re-train the regression models c_1 , c_2 , and c_3 on training dataset. We remove the chosen sample from the pool and repeat the process for each sample.

C. Regression Models

Most of our active learning algorithms use a regression model. The choice of regression model has a substantial impact on the performance of the active learning algorithm. We experimented with various machine learning regression techniques in the active learning algorithms and finalized the below three based on performance.

- 1) Random Forest Regression (RF): Random Forest [22] (RF) is an ensemble learning algorithm that uses multiple decision trees and combines their output to give final predictions. The benefit of using ensemble learning methods is that they are not dependent on the prediction of only one model. However, it considers the prediction of multiple models before giving the final prediction and thus has been effective in solving many complex regression problems [35], [36].
- 2) XGBoost (XGB): XGB [24] is also a decision tree-based ensemble learning algorithm that uses gradient boosting. XGB uses an optimized gradient descent algorithm. It employs regularization to avoid overfitting. We used XGB because it has given good performance in prior studies in performance prediction [25], [15], [26].
- 3) Extra Trees Regression (ETR): ETR [23] is an ensemble algorithm similar to RF. A significant difference between ETR and RF lies in selecting cut points. RF chooses the optimum split whereas ETR chooses it randomly. We have selected ETR based on its performance in prior works on performance prediction [25], [26].

D. Workflow for Evaluation

We used the TAU profiler [37] to determine the computation and communication times spent in MPI applications [38]. TAU profiler generates a summary of the application's performance, i.e., the time spent by the application in each MPI function.

We extract the communication and computation timings from the application's TAU profiles (generated for every rank and thread). The statistical methods use active learning methods to build the training dataset. The active learning algorithm selects the samples to be included in the training dataset. We obtain the labels only for the selected samples, not for every sample in the unlabeled pool. When an active learning algorithm requests a new sample, we schedule it on the cluster/supercomputer and obtain its execution time.

The entire dataset is divided into two sections. 20% of the data is kept for evaluating the trained model. The remaining data is used for training purposes. We train using varying percentages of this 80% pool as shown in Section IV-C. All AL algorithms are applied only to this training pool. The AL algorithm generates the final training dataset, and the regression model is trained using this dataset. Finally, the trained model is applied to the test dataset and its predictions are compared to the actual times of the test dataset. Note that the training data set of each AL algorithm may not be exactly same because every AL algorithm selects the next sample based on a different criteria. However, we ensured that all the algorithms are tested on the same test data for fair comparison.

IV. EXPERIMENTS AND RESULTS

This section explains the details of our experiments and the evaluation platform. We present the results obtained from various performance models on several parallel applications.

A. Benchmarks and Applications

We used five scientific applications and benchmarks to validate our performance models, namely HPCG v3.1 [39], [27], LULESH 2.0 [40], and the Mantevo mini benchmarks – miniAMR [41], miniFE [42], and miniMD [43]. We now briefly describe the applications and the input parameters (problem size, node and process counts) for these.

- HPCG: High-Performance Conjugate Benchmark (HPCG) performs sparse matrix calculations to model data access patterns of large applications. HPCG consists of a 3D problem space in x, y, and z dimensions. We used 2, 4, 8, and 16 nodes (processes per node = 32) and problem sizes of 64, 128, 200, and 256 in all three dimensions.
- LULESH: Livermore Unstructured Lagrangian Explicit Shock Hydrodynamics (LULESH) performs a hydrodynamics stencil calculation using MPI. LULESH takes a single parameter s to define the problem size and constructs a problem with s^3 atoms to solve. We varied s from 80 to 200 with a step size of 10. LULESH requires cubic number of MPI ranks. We used (2,32), (3,9), (4,16), (5,25), (6,36), (8,8), (8,27), (9,24), (12,18), and (16,32) as combinations of nodes and processes per node.
- miniAMR: miniAMR is a proxy application for adaptive mesh refinement. This is used for efficiently solving partial differential equations. miniAMR uses a 3D mesh structure, and the input parameters n_x , n_y , and n_z define the number of blocks in x, y, and z dimensions. We used node sizes 2,

4, 8, and 16 (processes per node = 32) and problem sizes of 4, 8, 12, and 16 in all three dimensions.

- **miniFE:** miniFE is a mini-application for implicit unstructured finite element code. The physical domain of miniFE is a modifiable three-dimensional box. We used 2, 4, 8, and 16 nodes (processes per node = 32) and problem sizes 128, 200, 256, 300, 384, and 512 in all three dimensions.
- **miniMD:** miniMD is a miniature version of the LAMMPS molecular dynamics (MD) application. miniMD allows us to define the problem size, atom density, temperature, and other variables. miniMD's feature set is somewhat limited; it only supports one form of pair interaction (Lennard Jones). We used 2, 4, 8, and 16 nodes (processes per node = 32) and problem sizes 128, 200, 256, 300, 384, and 512 in all three dimensions.

Table I shows the sizes of parameter space, training pool, and test dataset for all applications in our experiments. We have reserved 20% parameter space as test data and used the remaining 80% space as the training pool in our active learning algorithms. We have differing total number of samples because

TABLE I: Parameter space of MPI applications

Application	Total samples	Training data size	Test data size
HPCG	259	207	52
LULESH	248	198	50
miniAMR	256	204	52
miniFE	864	691	173
miniMD	864	691	173

the input parameter space is different for every application. The parameter restrictions and large runtimes restricts the parameter spaces of HPCG, LULESH and miniAMR. In LULESH, there is a single parameter for data size and its combination with the number of processes generates fewer samples. HPCG and miniAMR applications require large memory for large data sizes. This restricted the number of samples in parameter space. miniFE and miniMD ran successfully with many input values and thus have large parameter spaces.

B. Evaluation

We conducted our experiments on a 1.3 PFLOPS supercomputer, PARAM Sanganak, at the Indian Institute of Technology Kanpur, India. The compute nodes are interconnected through a 100 Gbps Mellanox (HDR) Infiniband network, and the system incorporates the Lustre parallel file system. We used a maximum of 16 nodes (Intel Xeon Cascade Lake processors) with 32 processes per node (a total of 512 processes). We used the Intel MPI compiler v2020. Our jobs ran in exclusive mode so that other jobs do not share on-chip resources of our jobs and thus exhibit lower node-level interference. However, the shared interconnect of the supercomputer does lead to interference with other jobs [44] and hence variation in runtimes. We scheduled approximately 2000 jobs to collect runtimes for prediction. We evaluate the performance of our regression models for performance prediction of parallel programs using Median Absolute Percentage Error (MdAPE) [45], Mean Absolute Percentage Error (MAPE) and R^2 score (coefficient of determination). R^2 score provides information about how

well the regression model approximates the actual data. A higher value shows a good fit for the regression model.

C. Results

We experimented with the models explained in Section III using the applications described in Section IV-A to predict the execution time. Our models use the active learning methods – Greedy sampling (Section III-A) and Query by committee (QBC) (Section III-B). We compared the performance of our models with uniform sampling [46]. We experimented with two different training dataset sizes – 40% and 60% of the data. Note that our goal is to evaluate accuracy of the models from fewer training samples such that the training cost is minimal. We present detailed observations for every application.

1) **HPCG 40% Training pool (83 samples):** Table II shows the values of Median Absolute Percentage Errors (MdAPE) and R^2 scores. Bold font in the left table shows the best performing regression algorithm for each of the three categories – greedy sampling, QBC, and uniform sampling. Among the greedy sampling techniques, G_x gives the best MdAPE (lowest) of 17.47 with the XGB model. QBC and uniform sampling achieve the best MdAPE (lowest) of 21.12 and 13.43, respectively, with ETR and XGB algorithms. G_x algorithm does not rely on the prediction to select the next data sample unlike the others (G_y , G_{xy} , and ALER). This may lead to poor performance of G_y , G_{xy} and ALER algorithms due to prediction errors with very few initial samples. We observe that G_x is best greedy sampling algorithm for all the three machine learning regression algorithms. This is because G_x and uniform sampling algorithms do not use predictions of machine learning model to build the dataset, and thus give the lowest error on this dataset. Also, there are fewer samples in 40% training pool, due to which G_y , G_{xy} do not get reliable predictions to build the dataset. We also observe that RF performs the worst.

TABLE II: HPCG results on 40% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	21.44	29.95	40.97	32.93	17.95
ETR	18.9	23.25	22.57	21.12	13.6
XGB	17.47	21.43	21.7	21.24	13.43

Median Absolute Percentage Errors

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	0.6	0.63	0.79	0.69	0.83
ETR	0.58	0.8	0.85	0.77	0.79
XGB	0.67	0.72	0.86	0.72	0.81

R^2 scores

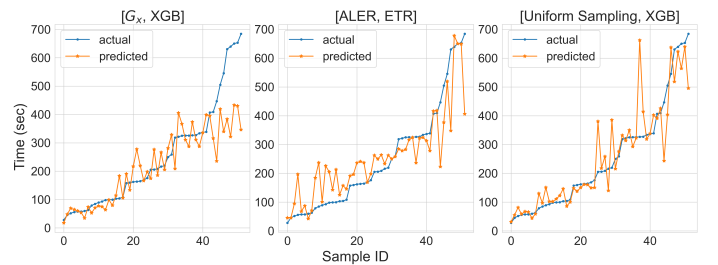


Fig. 1: HPCG: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 40% training pool

Figure 1 compares the actual and predicted execution times for the best greedy sampling, QBC and uniform sampling algorithms. X-axis represents the sampleID and Y-axis represents the time in seconds. We sorted the samples in increasing order of actual execution time. The number of test data points is shown in Table I. We observe that G_x and ALER show

high errors for the last few samples (after sampleID 40) compared to the earlier samples. These later samples have higher execution time compared to earlier samples and our models are uncertain on these samples. A possible reason of poor performance of G_x and ALER on the later samples is that the training dataset selected by the G_x and ALER do not contain enough number of samples with higher execution time. Uniform sampling performs better on these samples because training set of uniform sampling contains samples from all regions of training pool. However, uniform sampling has the highest maximum error of 102.81% as compared to 76.33% (maximum error) with G_x . We also observe higher errors with uniform sampling for some test data points in Figure 1. Thus the mean absolute percentage error (MAPE) for G_x and uniform sampling using XGB are 0.22 and 0.20 respectively, which shows similar overall performance with fewer training samples.

2) *HPCG 60% Training pool (125 samples)*: Table III shows the Median Absolute Percentage Errors (MdAPE) and R^2 scores. Bold font shows the best performing regression algorithm for greedy sampling, QBC, and uniform sampling. ETR and XGB algorithms perform better than RF. We observe that the QBC gives higher error compared to greedy and uniform sampling. This may be because the committee members are not able to select the subsequent sample correctly which does not result in a good dataset. All algorithms perform better with 60% training data as compared to the 40% pool, as observed from Tables II and III. The lowest errors of greedy sampling, QBC, and uniform sampling for 60% training pool are lower by 39.32%, 12.16% and 39.91% compared to 40% pool. The reason for this may be the increased number of samples, due to which the algorithms are able to derive the relation between the input and output. G_{xy} is able to perform better with more samples as compared to 40% pool. This may be due to better prediction of ML model to select diverse training samples based on both input and output domains. G_x and uniform with XGB both achieve 19% mean absolute percentage error.

TABLE III: HPCG results on 60% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling		Greedy Sampling			QBC (ALER)	Uniform Sampling	
	G_x	G_y	G_{xy}				G_x	G_y	G_{xy}			
RF	16.46	20.86	25.39	20.1	12.2	R^2 scores	RF	0.75	0.64	0.88	0.82	0.9
ETR	18.2	19.33	18.65	18.55	8.07		ETR	0.77	0.74	0.87	0.87	0.91
XGB	13.54	14.95	10.6	22.5	12.01		XGB	0.75	0.7	0.87	0.75	0.89
Median Absolute Percentage Errors												

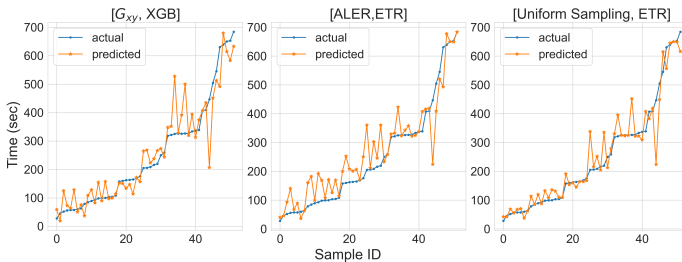


Fig. 2: HPCG: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 60% training pool

Figure 2 compares the actual (blue) and predicted (orange) execution times for the best Greedy sampling, QBC and uniform sampling algorithms. We observe that greedy sampling (G_{xy}) and ALER are not showing higher uncertainty on later

samples in test data (sampleID greater than 40) as was the case in 40% pool. The reason for this is that the dataset selected by these algorithm may contain enough number of samples that models are able to generalize the performance on these samples. Uniform sampling gives lowest error on this dataset. A possible reason for better performance of uniform sampling is the parameter space of HPCG. Uniform sampling selects samples uniformly from all regions of the parameter space, that may help in better training for HPCG. The active learning algorithms use different parameters to select the samples from the parameter space of the application. The performance of uniform sampling suggests that in HPCG parameter space, the information is spread uniformly across the parameter space and selecting various regions using active learning does not improve the accuracy.

3) *LULESH 40% Training pool (80 samples)*: Table IV shows the Median Absolute Percentage Errors (MdAPE) and R^2 . The best performing regression algorithm for greedy sampling, QBC, and uniform sampling is shown in bold. G_{xy} greedy sampling algorithm gives the best Median Absolute Percentage Error (MdAPE) of 6.48%. We observe that ETR algorithm performs better than the RF and XGB on this dataset. Although all the three algorithms are ensemble methods that combine various learners to produce the final output, a possible reason for better performance of ETR is the hyperparameter settings that help to learn the relationship better than other algorithms. We observe that G_{xy} algorithm performs better than G_x and G_y algorithms. This may be because G_{xy} uses both input and output domains to select the next sample whereas G_x and G_y select samples based on input or output domain. We also note that G_{xy} has high R^2 score which implies that it was able to learn the input-output relationship well, similar to uniform sampling. ALER only depends on the ML predictions to build the dataset. Thus ALER is not able to perform well with fewer samples in the 40% pool. However, note that the MdAPE for LULESH are relatively lower than HPCG even with 40% training data.

TABLE IV: LULESH results on 40% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling		Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}				G_x	G_y	G_{xy}		
RF	15.48	18.18	13.78	12.81	7.83	R^2 scores	0.91	0.83	0.95	0.87	0.97
ETR	8.36	11.99	6.48	13.15	7		0.95	0.9	0.95	0.94	0.98
XGB	13.87	13.71	10.61	9.59	7.51		0.94	0.9	0.97	0.97	0.98
Median Absolute Percentage Errors											

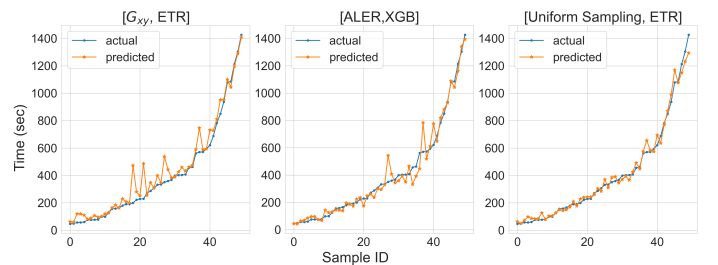


Fig. 3: LULESH: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 40% training pool

Figure 3 shows the comparison between actual (blue) and predicted (orange) execution times for the best-performing Greedy Sampling, QBC, and uniform sampling algorithms. G_{xy}

with ETR is best performing algorithm on this dataset. The error obtained by G_{xy} is 7.42% lower than uniform sampling and 27% lower than QBC. We also note that the greedy sampling algorithm does not exhibit high errors for large execution times as compared to HPCG with 40% training pool even though the maximum execution times of LULESH are $2\times$ higher. This may be because the parameter space of LULESH is smaller than HPCG; LULESH has one problem size parameter (s) whereas we use n_x , n_y and n_z in case of HPCG to characterize the problem size. Thus it may be easier to model LULESH.

4) *LULESH 60% Training pool (119 samples)*: Table V shows the Median Absolute Percentage Errors (MdAPE) and R^2 . The lowest Median Absolute Percentage Error (MdAPE) is 3.55 using G_x greedy sampling with ETR. This is 40% lower than the best MdAPE with uniform sampling. The lowest Mean Absolute Percentage Error (MAPE) is also obtained using G_x with ETR. Thus G_x is able to build a good dataset from the LULESH training pool. This is because G_x selects samples based on the diversity in the input domain unlike uniformly spaced samples in case of uniform sampling.

TABLE V: LULESH results on 60% training pool with statistical models (Bold font shows the best performing combination)

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	10.12	15.06	6.02	16.29	6.73
ETR	3.55	8.28	5.49	6.39	6.77
XGB	10	8.26	7.23	12.24	5.94

Median Absolute Percentage Errors

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	0.96	0.81	0.98	0.95	0.98
ETR	0.98	0.91	0.97	0.98	0.98
XGB	0.98	0.94	0.98	0.96	0.99

R^2 scores

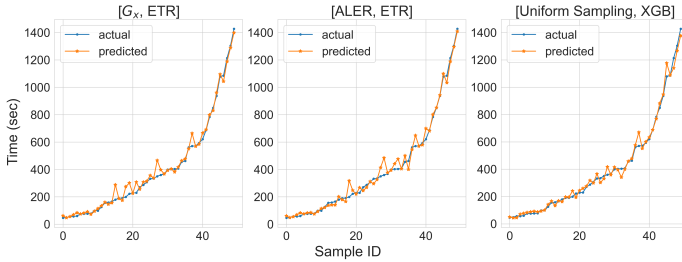


Fig. 4: LULESH: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 60% training pool

Figure 4 compares the actual (blue) and predicted (orange) execution time for the best-performing greedy sampling (G_x), QBC, and uniform sampling algorithms. We observe that all three techniques are able to predict the execution time well on the test dataset. The lowest error with 60% training pool for greedy sampling, QBC, and uniform sampling decreased by 45.21%, 33.36% and 15.14% as compared to 40% pool.

5) *miniAMR 40% Training pool (82 samples)*: Table VI shows the Median Absolute Percentage Errors (MdAPE) and R^2 scores. Uniform sampling does not depend on the prediction of ML models for selecting the following sample. Thus uniform sampling with XGB gives the lowest error on this dataset. G_x also does not depend on ML model prediction for sample selection, it is the best greedy sampling algorithm. The R^2 scores of greedy and uniform sampling are similar (less than 0.8), thus indicating that it is hard to learn the input-output relationship in miniAMR application.

TABLE VI: miniAMR results on 40% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	15.94	18.19	27.14	26.31	15.27
ETR	17.82	18	19.82	25.96	13.32
XGB	20.66	18.8	18	26.79	9.82

Median Absolute Percentage Errors

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	0.69	0.34	0.74	0.1163	0.73
ETR	0.52	0.09	0.61	-0.11	0.64
XGB	0.49	0.11	0.72	-0.59	0.64

R^2 scores

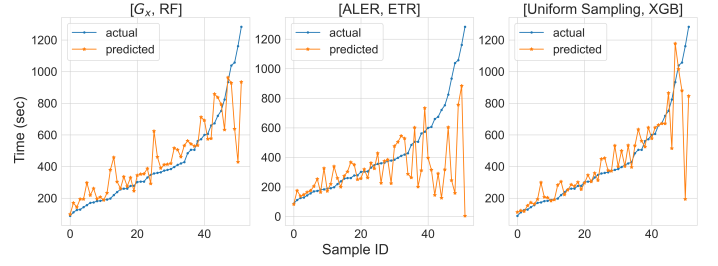


Fig. 5: miniAMR: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 40% training pool

Figure 5 compares actual (blue) and predicted (orange) execution times for best Greedy sampling, QBC, and uniform sampling algorithms. We observe that all models are unable to correctly predict the execution time for the samples that have higher execution time (after sampleID 45). We analyzed the relationship of input problem size parameters and found certain input-output relationships that may be hard to learn. For example, on 8 nodes with 32 processes per node, the actual execution time of a problem with dimensions $8 \times 12 \times 12$ is 419.55 seconds whereas the actual execution time of a problem with dimensions $12 \times 12 \times 8$ (same volume) is 824.29 seconds. However, the actual times of $12 \times 8 \times 16$ and $8 \times 16 \times 12$ (same volume) were similar (600 and 606 seconds) on 2 nodes (with 32 processes per node). It is possible that the domain decomposition of the domain played a significant role in this application and was hard to model. It is also hard to select good samples in such cases which is why greedy sampling algorithms perform worse than uniform sampling. We plan to investigate this in future.

6) *miniAMR 60% Training pool (123 samples)*: We observe from table VII that the QBC performs better than the greedy sampling and uniform sampling in this case. QBC gives 23.27% lower MdAPE than greedy sampling and 2.02% lower MdAPE than uniform sampling. QBC did not perform well with 40% training data because the ML models cannot efficiently learn the mapping with fewer samples. The performance of committee improved with more samples in the 60% pool.

TABLE VII: miniAMR results on 60% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	13.97	16.62	17.68	22.8	10.9
ETR	13.85	10.7	11.7	13.17	9.46
XGB	13	11.77	12.9	8.21	8.38

Median Absolute Percentage Errors

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	0.74	0.71	0.78	0.75	0.75
ETR	0.53	0.46	0.54	0.62	0.6
XGB	0.62	0.63	0.48	0.76	0.53

R^2 scores

Figure 6 compares actual and predicted execution times for all three. We observe that both greedy sampling and uniform sampling are unable to correctly predict the execution time for samples with higher execution time (sampleID greater than 45). ALER is performing better than other algorithms for these samples. We obtain lower errors with all algorithms in general on this dataset. The errors obtained on the 60% pool by greedy sampling, QBC, and uniform sampling are 32.87%, 68.37%, and 14.66% lower than the 40% pool. This shows

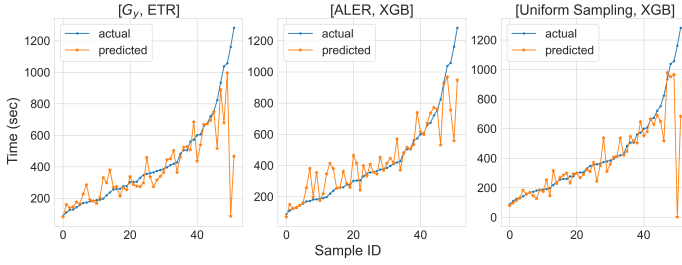


Fig. 6: miniAMR: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 60% training pool

that complex input-output relationship may be learnt better with higher number of samples and carefully selected samples.

7) *miniMD 40% Training pool (276 samples)*: We observe in Table VIII that greedy sampling G_x and uniform sampling give similar lowest MdAPE. This may be because G_x selects diverse samples based on the input domain and algorithms G_x and uniform sampling algorithms do not rely on ML model predictions for selecting the subsequent samples in the dataset. The average error (MAPE) using G_x with ETR is 20.59% whereas uniform sampling with ETR gives 22.87% MAPE. We observe that the errors on miniMD dataset are slightly higher than HPCG, miniAMR, LULESH, even though there are more samples (276) in this dataset. This is probably due to complex parameter space. Figure 7 shows the comparison between actual (blue) and predicted (orange) execution time for best-performing Greedy Sampling, QBC, and uniform sampling algorithms. We observe considerable overlap between the blue and orange curves in case of G_x .

TABLE VIII: miniMD results on 40% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}			G_x	G_y	G_{xy}		
RF	17.98	35.34	44.54	34.61	21.38	0.96	0.77	0.93	0.82	0.93
ETR	16.53	29.18	35.19	28.04	16.84	0.91	0.84	0.93	0.86	0.95
XGB	18.19	23.72	26.96	20.71	16.46	0.9	0.88	0.94	0.84	0.93

Median Absolute Percentage Errors

R^2 scores

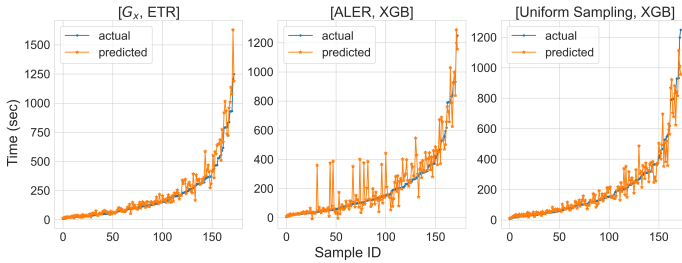


Fig. 7: miniMD: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 40% training pool

8) *miniMD 60% Training pool (414 samples)*: G_x greedy sampling algorithm with ETR gives the lowest MdAPE (12.62%) on this dataset. The best MdAPE of greedy sampling is 3.14% lower than uniform sampling and 22.76% lower than the QBC. The MdAPE errors are lower with 60% training data. Figure 8 compares actual and predicted execution time for best Greedy sampling, QBC, and uniform sampling algorithms. The average error (MAPE) using G_x with ETR is 17.38% whereas uniform sampling with ETR gives 19.07% MAPE. The maximum error with uniform sampling is 3.4% lower than the maximum error with G_x .

TABLE IX: miniMD results on 60% training pool with statistical models

	Greedy Sampling			QBC (ALER)	Uniform Sampling	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}			G_x	G_y	G_{xy}		
RF	14.78	25.93	23.96	26.42	17.24	0.96	0.84	0.96	0.9	0.95
ETR	12.62	22.22	20.07	21.99	13.03	0.95	0.87	0.96	0.92	0.95
XGB	14.6	14.97	17.54	16.34	13.85	0.96	0.94	0.944	0.94	0.96

Median Absolute Percentage Errors

R^2 scores

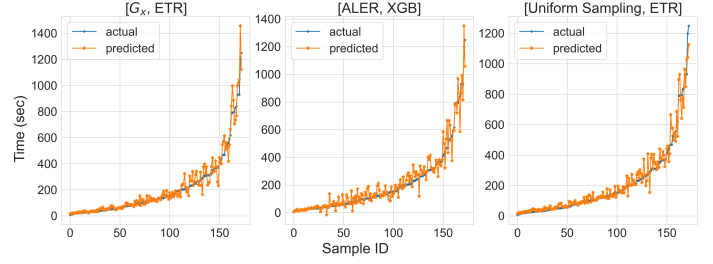


Fig. 8: miniMD: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 60% training pool

9) *miniFE 40% Training pool (276 samples)*: Table X shows the MdAPE and R^2 scores obtained in our experiments. Bold font in the table shows the best-performing algorithm from the three categories – greedy sampling, QBC, and uniform sampling. The error values for miniFE are higher than other applications. The lowest MdAPE (32.18%) is obtained with uniform sampling and ETR regression. However, uniform sampling and other sampling techniques using the various regression approaches exhibit low R^2 scores, which show that ML models do not effectively learn the mapping between input and output domains even with hyperparameter tuning (as done in all cases).

TABLE X: miniFE results on 40% training pool with statistical models (Bold font shows the best performing combination)

	Greedy Sampling			QBC (ALER)	Uniform Sampling	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}			G_x	G_y	G_{xy}		
RF	46.56	59.7	47.78	60.6	44.15	-1.64	-0.02	0.15	-0.24	-0.03
ETR	45.14	40.71	38.99	43.73	32.18	-1.39	-0.1	0.06	-0.75	-0.47
XGB	53.1	84.12	118.43	90.34	64.55	-3.2	-1.4	-2.16	-3.17	-1.74

Median Absolute Percentage Errors

R^2 scores

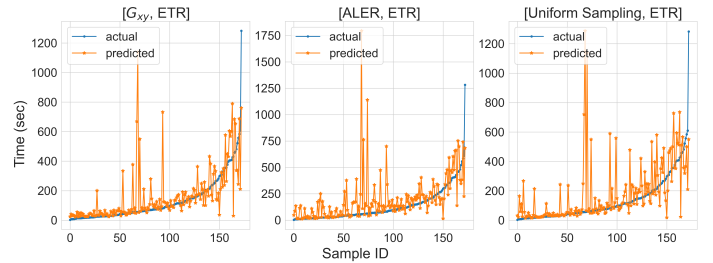


Fig. 9: miniFE: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 40% training pool

Figure 9 compares actual and predicted execution time for best Greedy sampling, QBC, and uniform sampling algorithms. We observe quite a few regions with high errors (example, around sampleID 60 - 70). We observed some crucial behavior based on the problem domain. For example, a problem size of $384 \times 128 \times 512$ takes 46.86s on 256 processes (8 nodes, 32 processes per node) whereas problem size of $512 \times 384 \times 128$ takes 97.71s on 256 processes (8 nodes, 32 processes per node). Similarly, we observed that a problem size of $200 \times 300 \times 256$ takes 121.96s on 128 processes (4 nodes, 32 processes per node) whereas problem size of $256 \times 300 \times 200$ takes 73.09s on 128 processes (4 nodes, 32 processes per node). It is possible

that this behaviour was hard to learn from just a few hundred samples for a large parameter space (see Section IV-A).

10) *miniFE 60% Training pool (414 samples)*: Table VII shows the values of Median Absolute Percentage Errors (MdAPE) and R^2 scores obtained in our experiments for this dataset. The error values are still quite high but lower than the 40% training pool. The ML models perform better with the increase in the number of samples. However, the R^2 scores are still low. The lowest value of MdAPE (25.94%) is obtained using uniform sampling with ETR. This MdAPE is 19.39% lower than the MdAPE on 40% training pool with uniform sampling.

TABLE XI: miniFE results on 60% training pool with statistical models (Bold font shows the best performing combination)

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	42.01	48.41	41.73	43.91	39.35
ETR	34.06	36.73	35.89	32.25	25.94
XGB	57.12	78.48	73.099	81.77	52.06

Median Absolute Percentage Errors

	Greedy Sampling			QBC (ALER)	Uniform Sampling
	G_x	G_y	G_{xy}		
RF	-0.4	0.122	0.27	0.01	0.02
ETR	-0.37	0.06	0.12	-0.09	-0.45
XGB	-2.47	-0.9	-1.75	-1.79	-1.06

R^2 scores

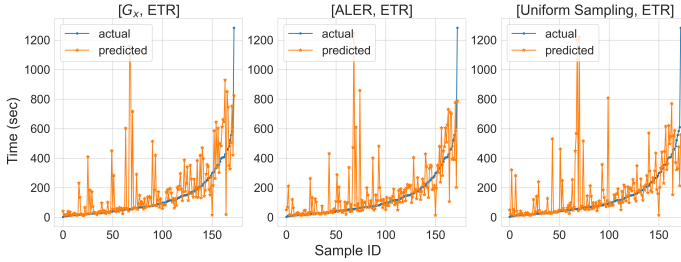


Fig. 10: miniFE: actual and predicted times for best performing Greedy sampling, QBC and uniform sampling on 60% training pool

Figure 10 compares actual (blue) and predicted (orange) execution times for best Greedy sampling, QBC, and uniform sampling algorithms. The values of MdAPE and R^2 scores on the miniFE training pool highlight the challenging parameter space of miniFE. Even with significantly larger number of samples (414 in 60% training pool), the machine learning algorithms predict execution times with high errors. We use our active learning algorithms to build the dataset from the parameter space of miniFE, and this selected dataset is not a good representative of the parameter space. The machine learning algorithms are unable to learn the mapping between input and output domains. We present a summary of the results next.

D. Discussion of Results

We presented detailed results of statistical models for all the applications in the previous subsection. We have experimented with two different training pool sizes for all applications. The results for 40% and 60% pools were presented here. We observe that in all the applications, the lowest MdAPE decreases as we increase the number of samples in the training pool. This happens because ML models are able to derive relationship between input and output domains with large number of samples. The 40% training pool of the applications have less number of samples, and we have seen that with less number of samples algorithms like G_y , G_{xy} , and ALER do not give good performance because it is difficult to build good dataset with

prediction on less number of samples. The algorithms like G_x and uniform sampling do not rely on the predictions of ML model to build the dataset, and these give best performance generally on 40% training pool. As we increases the number of samples in the 60% pool, the algorithms G_y , G_{xy} , and ALER improve their performance. The G_{xy} generally performs better in 60% pool because it uses both input and output domains to build the dataset. We also observed that ALER (QBC) never significantly outperforms the others.

Random forest regression performed worse than Extra trees regression and Extreme gradient boosting. We also observed that with just a few hundred training samples, the best MdAPE of most algorithms were below 15% for HPCG, LULESH, miniAMR and miniMD. miniFE application was the hardest to model and exhibited 25% median absolute percentage error even with 60% training data set. miniMD was the next application that was hard to learn for the ML models. However, we note that we used only 414 training samples for miniMD and miniFE. This is just 6% of the number of samples (uniformly selected) in a prior work on machine learning based performance prediction [25]. Here, the authors used 864, 6665, and 1620 training samples for miniAMR miniFE miniMD applications as compared to 123, 414, and 414 maximum training samples used in our case. Some of the skewed input configurations such as $512 \times 256 \times 200$ gave the highest errors in case of miniMD and miniFE. We may also need to add some more input parameters based on the domain decomposition. More sample data may further improve the performance of these. However, our goal was to evaluate the performance of sampling techniques for active learning so that we can select the next sample in an informed way. Fewer samples are selected and the labels (actual runtimes) are collected only for those samples, thereby leading to small training times. We observed that G_x and uniform sampling performs good along with ETR and XGB regression algorithms for most of the applications that we considered.

V. CONCLUSIONS AND FUTURE WORK

In this work, we have studied and presented statistical modeling approaches for the performance modeling of parallel applications. Statistical methods are generally based on classical machine learning approaches, which require a large amount of training data for better performance. We developed statistical models that use active learning techniques to achieve good performance using fewer training samples. Our statistical models with greedy sampling predict the execution times of five scientific applications – HPCG, LULESH, miniAMR, miniMD, and miniFE with MdAPE of 10.60, 3.55, 8.21, 12.62 and 32.25% respectively on up to 512 cores on a large-scale parallel system. Uniform sampling performs better in case of miniFE application.

Our statistical model results are comparable with the results presented by Malakar et al. [25] for miniAMR, miniFE and miniMD. However, they have used 864, 6665, and 1620 samples in the parameter space for miniAMR, miniFE and miniMD respectively. Our active learning methods have used only 30%,

12% and 12% parameter space for these three applications respectively. The statistical models in [25] achieve best MdAPE in 15-20% range. Our models achieve better performance with very less parameter space.

In future, we would first like to develop hybrid statistical and analytical modeling. We would also like to predict in an application-agnostic way such that training per application per system is not required or minimally required. We will also investigate the source of higher errors in case of certain applications such as miniMD and miniFE.

VI. ACKNOWLEDGEMENTS

We thank Indian Institute of Technology Kanpur for providing access to PARAM Sanganak supercomputer.

REFERENCES

- [1] L. Clarke and et al., "The MPI message passing interface standard," in *Programming environments for massively parallel distributed systems*. Springer, 1994, pp. 213–218.
- [2] M. M. Tikir, M. A. Laurenzano, L. Carrington, and A. Snavey, "PSINS: An open source event tracer and execution simulator for MPI applications," in *European Conference on Parallel Processing*, 2009, pp. 135–148.
- [3] T. Cornebize, A. Legrand, and F. C. Heinrich, "Fast and faithful performance prediction of MPI applications: the HPL case study," in *2019 IEEE International Conference on Cluster Computing*.
- [4] W. Chen and et al., "LogGPO: An accurate communication model for performance prediction of MPI programs," *Science in China Series F: Information Sciences*, vol. 52, pp. 1785–1791, 2009.
- [5] W. Zhang, A. M. Cheng, and J. Subhlok, "Dwarfcode: A performance prediction tool for parallel applications," *IEEE Transactions on Computers*, vol. 65, no. 2, pp. 495–507, 2015.
- [6] J. Flores-Contreras and et al., "Performance prediction of parallel applications: a systematic literature review," *The Journal of Supercomputing*, vol. 77, no. 4, pp. 4014–4055, 2021.
- [7] V. S. Adve and M. K. Vernon, "Parallel program performance prediction using deterministic task graph analysis," *ACM Transactions on Computer Systems (TOCS)*, vol. 22, no. 1, pp. 94–136, 2004.
- [8] R. Z. Khan, A. Q. Ansari, and K. Qureshi, "Performance prediction for parallel scientific application," *Malaysian Journal of Computer Science*, vol. 17, no. 1, pp. 65–73, 2004.
- [9] D. Culler and et al., "LogP: Towards a realistic model of parallel computation," in *Proceedings of the fourth ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, 1993, pp. 1–12.
- [10] A. Alexandrov and et al., "LogGP: Incorporating long messages into the LogP model one step closer towards a realistic model for parallel computation," in *Proceedings of the seventh annual ACM symposium on Parallel algorithms and architectures*, 1995, pp. 95–105.
- [11] Y. C. Tay, "Analytical performance modeling for computer systems," *Synthesis Lectures on Computer Science*, vol. 7, no. 1, pp. 1–171, 2018.
- [12] D. Sundaram-Stukel and et al., "Predictive analysis of a wavefront application using LogGP," in *Proceedings of ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, 1999, pp. 141–150.
- [13] L. Adhianto and et al., "Performance modeling of communication and computation in hybrid MPI and OpenMP applications," *Simulation Modelling Practice and Theory*, vol. 15, no. 4, pp. 481–491, 2007.
- [14] J. Sun and et al., "Automated performance modeling of HPC applications using machine learning," *IEEE Transactions on Computers*, vol. 69, no. 5, pp. 749–763, 2020.
- [15] S. Hunold and et al., "Predicting MPI collective communication performance using machine learning," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2020, pp. 259–269.
- [16] K. Singh and et al., "Predicting parallel application performance via machine learning approaches," *Concurrency and Computation: Practice and Experience*, vol. 19, no. 17, pp. 2219–2235, 2007.
- [17] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*. Springer, 2006, vol. 4, no. 4.
- [18] B. Settles, "Active learning literature survey," 2009.
- [19] D. Wu, "Pool-based sequential active learning for regression," *IEEE transactions on neural networks and learning systems*, vol. 30, no. 5, pp. 1348–1359, 2018.
- [20] D. Wu and et al., "Active learning for regression using greedy sampling," *Information Sciences*, vol. 474, pp. 90–105, 2019.
- [21] N. Fazakis, G. Kostopoulos, S. Karlos, S. Kotsiantis, and K. Sgarbas, "An active learning ensemble method for regression tasks," *Intelligent Data Analysis*, vol. 24, no. 3, pp. 607–623, 2020.
- [22] V. Svetnik, A. Liaw, C. Tong, J. C. Culberson, R. P. Sheridan, and B. P. Feuston, "Random forest: a classification and regression tool for compound classification and QSAR modeling," *Journal of chemical information and computer sciences*, vol. 43, no. 6, pp. 1947–1958, 2003.
- [23] "Extra Tree Regressor," <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesRegressor.html>, accessed: 2022-05-20.
- [24] T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen et al., "Xgboost: extreme gradient boosting," *R package version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2015.
- [25] P. Malakar and et al., "Benchmarking machine learning methods for performance modeling of scientific applications," in *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2018, pp. 33–44.
- [26] M. Agarwal and et al., "Active Learning-based Automatic Tuning and Prediction of Parallel I/O Performance," in *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, pp. 20–29.
- [27] HPCG benchmark. [Online]. Available: <https://github.com/hpcg-benchmark/hpcg>
- [28] LULESH benchmark. [Online]. Available: <https://github.com/LLNL/LULESH>
- [29] A. Snavey, L. Carrington, N. Wolter, J. Labarta, R. Badia, and A. Purkayastha, "A framework for performance modeling and prediction," in *SC'02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing*. IEEE, 2002, pp. 21–21.
- [30] N. Ding and et al., "Using hardware counter-based performance model to diagnose scaling issues of HPC applications," *Neural Computing and Applications*, vol. 31, no. 5, pp. 1563–1575, 2019.
- [31] D. Yokelson, M. R. J. Charest, and Y. W. Li, "Hpc application performance prediction with machine learning on new architectures," in *Proceedings of the 2023 on Performance Engineering, Modelling, Analysis, and Visualization Strategy*, 2023.
- [32] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big data*, vol. 3, no. 1, pp. 1–40, 2016.
- [33] Y. Wang and Q. Yao, "Few-shot learning: A survey," 2019.
- [34] R. Burridge and et al., "Active learning for regression based on query by committee," in *International Conference on Intelligent Data Engineering and Automated Learning*. Springer, 2007, pp. 209–218.
- [35] X. Zhou, X. Zhu, Z. Dong, W. Guo et al., "Estimation of biomass in wheat using random forest regression algorithm and remote sensing data," *The Crop Journal*, vol. 4, no. 3, pp. 212–219, 2016.
- [36] Y. Li, C. Zou, M. Bercebar, E. Nanini-Maury, J. C.-W. Chan, P. Van den Bossche, J. Van Mierlo, and N. Omar, "Random forest regression for online capacity estimation of lithium-ion batteries," *Applied energy*, vol. 232, pp. 197–210, 2018.
- [37] TAU profiling tool. [Online]. Available: <https://www.cs.uoregon.edu/research/tau/home.php>
- [38] "MPICHv3.3," <https://github.com/pmodels/mpich/tree/v3.3.2>, accessed: 2022-05-20.
- [39] J. Dongarra, M. A. Heroux, and P. Luszczek, "Hpcg benchmark: a new metric for ranking high performance computing systems," *Knoxville, Tennessee*, vol. 42, 2015.
- [40] I. Karlin, J. Keasler, and J. R. Neely, "Lulesh 2.0 updates and changes," Tech. Rep., 2013.
- [41] A. Sasidharan and M. Snir, "MiniAMR - A Miniapp for Adaptive Mesh Refinement," Tech. Rep., 2016.
- [42] A. B. Williams, "Cuda/GPU version of miniFE mini-application." Sandia National Lab., Tech. Rep., 2012.
- [43] P. Crozier and S. Plimpton, "miniMD v. 1.0," Tech. Rep., 2009.
- [44] A. Bhatlele, K. Mohror, S. H. Langer, and K. E. Isaacs, "There Goes the Neighborhood: Performance Degradation Due to Nearby Jobs," in *Proceedings of the 2013 ACM/IEEE conference on Supercomputing*.
- [45] J. S. Armstrong and F. Collopy, "Error measures for generalizing about forecasting methods: Empirical comparisons," *International journal of forecasting*, vol. 8, no. 1, pp. 69–80, 1992.
- [46] S. Arnold, G. Dhillion, A. Ravichandran, and S. Soatto, "Uniform Sampling over Episode Difficulty," *Advances in Neural Information Processing Systems*, vol. 34, pp. 1481–1493, 2021.